AI Automation for Developers

https://github.com/DataForScience/AIAutomation



Bruno Gonçalves
Data For Science, Inc
www.data4sci.com/newsletter
data4sci.substack.com

Copyright 2026 Pearson. All rights reserved.

Contacts



Bruno Gonçalves is an author, public speaker, corporate trainer, and consultant specializing in Generative AI, Blockchain Analytics, and Machine Learning. He founded Data For Science, Inc in 2009 to help individuals and companies solve their data driven problems.

Bruno Gonçalves

https://data4sci.com

info@data4sci.com

https://data4sci.com/call

https://www.linkedin.com/in/bmtgoncalves/

Table of Contents:

1 Structured Extraction

2 Summarization

3 Agents

4 Chatbot



Table of Contents:

1 **Structured Extraction**

2 **Summarization**

3 **Agents**

4 **Chatbot**



Summarization Project Overview

- Goal: Turn unstructured job-description text into a clean, validated, downstream-friendly schema using Claude + LangChain + Pydantic.

- What we'll build
  - 1. Pydantic `JobPosting` schema with realistic validators
  - 2. LangChain extraction chain wired to `claude-opus-4-5` (or any current model)
  - 3. JSON-validation loop that self-heals on schema failures
  - 4. Prompt guardrails that refuse non-job-posting inputs
  - 5. A small evaluation pass + visual sanity check

- Dataset: https://huggingface.co/datasets/jacob-hugging-face/job-descriptions

LangChain Deep Dive

- **LangChain** composes LLM workflows from small primitives
- Core building blocks:
  - **ChatModel**: wraps Claude / OpenAI / etc with one API
  - **PromptTemplate**: typed, reusable prompts with variables
  - **OutputParser**: turns text into typed Python objects
  - **Runnable | pipe**: compose with the | operator into chains
  - **Tool**: typed function the model can call
- All immutable, all introspectable, all batch-able

LangChain: minimal chain

```python
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate

llm = ChatAnthropic(model="claude-opus-4-5", temperature=0)
prompt = ChatPromptTemplate.from_messages([
    ("system", "Extract the company name from the text."),
    ("user", "{text}"),
])
chain = prompt | llm
chain.invoke({"text": "Acme Corp announced earnings today..."})
```

Pearson

LangChain: scaling out

- **invoke**: single call
- **batch**: parallel calls: same chain, list of inputs
- **stream**: token-by-token output for live UIs
- **ainvoke / abatch / astream**: async equivalents
- Why this matters in production:
  - batch on 10 items with **max_concurrency=5** gives you 5× throughput for free
  - Same code path for one item or ten thousand

LangChain: ecosystem

- **langchain-core**: primitives, no provider deps
- **langchain-anthropic / -openai / …**: model bindings
- **langchain-text-splitters**: recursive, semantic chunkers
- **langgraph**: stateful graphs for agents and human-in-the-loop
- **langsmith**: tracing, evals, prompt versioning
- **Pick small pieces. Don't import the kitchen sink.**

Pydantic schema planning

- **Pydantic** is the typed schema layer for structured extraction
- Three principles when designing an extraction schema:
  - **Explicit nullability:** anything optional is **Optional[...] = None**, not a sentinel string
  - **Constrain enums:** **Literal['junior','mid','senior']** prevents free-text drift
  - **Validators are part of the prompt:** their error messages flow back to the model on retry

Schema as contract

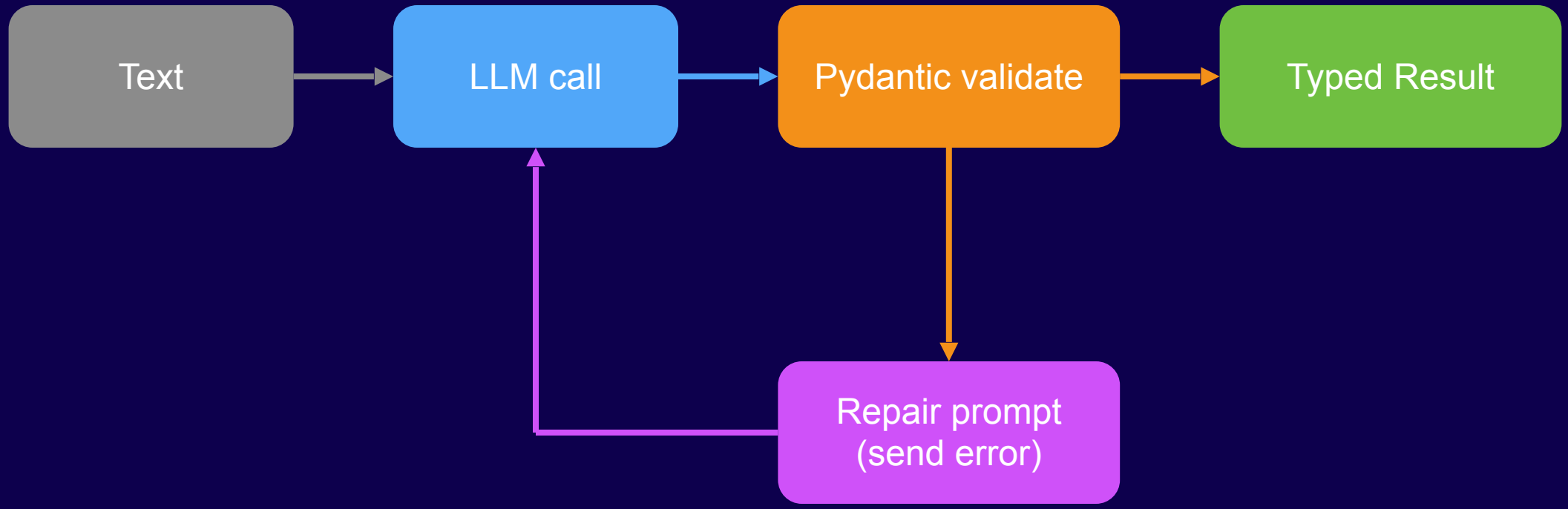
- **model_json_schema()** gives you a JSON Schema you can hand to:
  - the model: **PydanticOutputParser** injects it into the prompt
  - downstream consumers: DB schemas, OpenAPI specs, TypeScript types
  - evals: gold-set validation against the same schema

```
{
  "type": "object",
  "properties": {
    "title":     {"type": "string"},
    "remote":    {"enum": ["remote", "hybrid", "onsite"]},
    "skills":    {"type": "array", "items": {"type": "string"}},
    "salary":    {"$ref": "#/definitions/SalaryRange"}
  },
  "required": ["title", "is_job_posting"]
}
```

JSON validation loop

- Models occasionally produce malformed JSON or fail schema constraints
- **The cheapest fix is a one-shot self-repair:**
  - Catch the **ValidationError**
  - Re-prompt with the **exact validator message,** the broken JSON, and the schema
  - Parse again. Succeeds ~85% of the time on first retry
- Cap retries at 1–2; fail loudly after that
  - Log every failure
  - The long tail tells you where your schema is wrong

JSON validation loop: flow



- On success, it outputs a typed object.
- On **ValidationError**, it uses the repair prompt to feed the error back.
- Cap retries.

Pearson

Prompt guardrail patterns

- **Three guardrails that prevent ~80% of production incidents:**
  - **Input typing:** add an **is_job_posting: bool** field. The model can say "this isn't my problem" instead of hallucinating.
  - **Source-grounding:** ”only extract values that appear verbatim or are unambiguously implied". Cuts hallucinations sharply.
  - **Prompt-injection defense:** wrap source in delimiters and instruct the model to treat it as **data, not instructions**.

Structured Extraction

https://github.com/DataForScience/AIAutomation

Copyright 2026 Pearson. All rights reserved.

Table of Contents:

1 Structured Extraction

2 Summarization

3 Agents

4 Chatbot



Summarization Project Overview

- Goal: Take a stack of arXiv papers and produce, per paper:
  - Category label (router output),
  - Short executive summary, and
  - List of concrete action items a researcher should take after reading.

- We'll cover the four canonical patterns of LLM summarization:
  - Map-reduce for documents that don't fit in context
  - Action-item templates with structured output
  - Router prompts to dispatch to specialist summarizers
  - Misclassification audit to catch the router when it lies

- Dataset: https://huggingface.co/datasets/ccdv/arxiv-summarization

Pearson

Summarization

- Summaries are compressions
- Three common failure modes:
  - Hallucinated facts not in the source
  - Genre mismatch: applied summary of a theory paper
  - Length collapse: five sentences become one
- Each has a distinct mitigation pattern

Four canonical patterns

- **Map-reduce:** For documents that don't fit in context
- **Action-item templates:** When the summary should drive a downstream action
- **Router prompts:** Dispatch to a specialist by document type
- **Misclassification audit:** Second pass to catch bad routes & hallucinations

Four canonical patterns

- **Map-reduce:** For documents that don't fit in context
- **Action-item templates:** When the summary should drive a downstream action
- **Router prompts:** Dispatch to a specialist by document type
- **Misclassification audit:** Second pass to catch bad routes & hallucinations

Action-item templates

- A summary is rarely the deliverable. Somebody needs to do something with it
- Make the action explicit. Use a typed schema:
  - **PaperBrief** → summary + **List[ActionItem]** + **List[OpenQuestion]**
  - ActionItem.action: Literal['read-cited-paper', 'replicate-experiment', 'try-method', 'flag-for-review', 'discuss-with-team', 'no-action']
- **Concrete beats vague. "Replicate Table 3 on CIFAR-10" beats "investigate further".**

Action-item: the schema

```python
class ActionItem(BaseModel):
    action: Literal[
        'read-cited-paper', 'replicate-experiment',
        'try-method', 'flag-for-review',
        'discuss-with-team', 'no-action',
    ]
    detail: str = Field(..., description='Concrete next step, < 200 chars')
    priority: Literal['low', 'medium', 'high']

class PaperBrief(BaseModel):
    summary: str
    action_items: List[ActionItem] = Field(default_factory=list, max_length=5)
    open_questions: List[str] = Field(default_factory=list, max_length=3)
```

Pearson

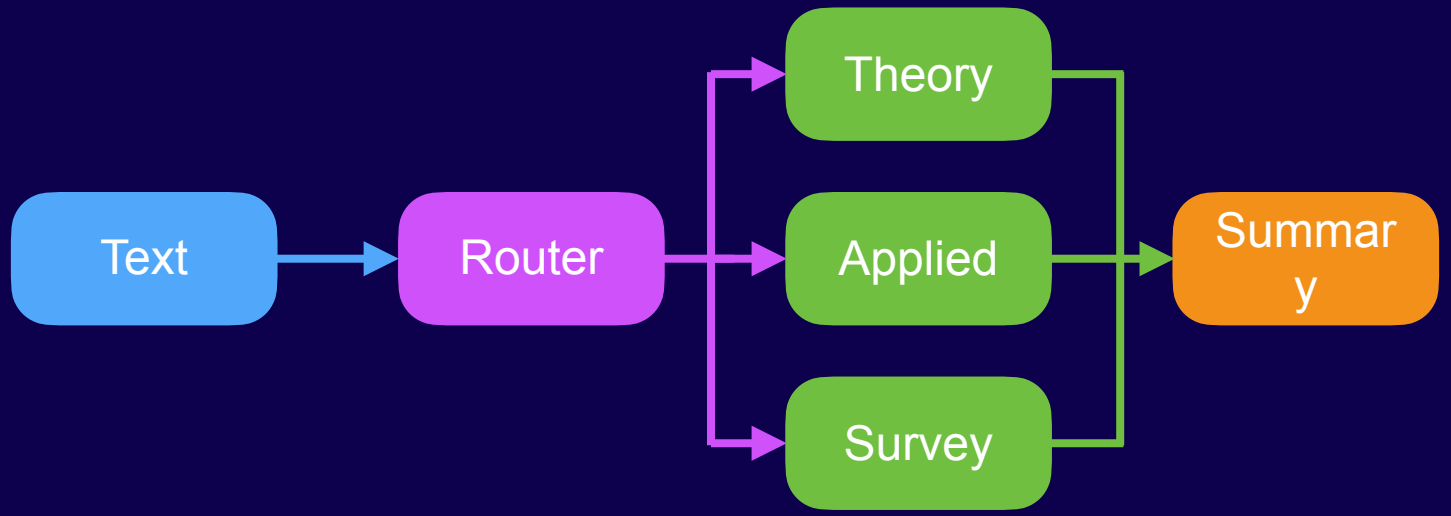
Wiring action items into the chain

- **PydanticOutputParser(pydantic_object=PaperBrief)**
  - injects format instructions into the prompt
  - parses the model's response into a typed Python object

- The downstream consumer doesn't see JSON: it sees objects
  - **brief.action_items[0].priority == 'high'** is now a real, type-checked Python expression

Router prompts

- Classify first, then dispatch to a specialist summarizer
- Why bother:
  - **A theory paper summarised with an applied template fabricates datasets**
  - Survey papers need a different shape (taxonomy + representative works)
  - **Routing classifications cost ~$0.001: cheaper than a single bad summary**



Router: the chain

```python
ROUTER = ChatPromptTemplate.from_messages([
    ("system", "Classify the paper as one of: theory, applied, "
               "survey. Output exactly one lowercase word."),
    ("user", "{abstract}"),
])
router_chain = ROUTER | llm | StrOutputParser()

specialists = {
    "theory":  THEORY_PROMPT  | llm | StrOutputParser(),
    "applied": APPLIED_PROMPT | llm | StrOutputParser(),
    "survey":  SURVEY_PROMPT  | llm | StrOutputParser(),
}

def route_and_summarize(text):
    cat = router_chain.invoke({"abstract": text[:2000]}).lower()
    if cat not in specialists:
        cat = "applied"          # safe default
    return specialists[cat].invoke({"summary": map_reduce(text)})
```

Pearson

Router design: practical tips

- **Output exactly one word**: easier to validate than free-form classification
- **Have a safe default**: when the router hallucinates, fall through to your least-bad specialist
- **Use only the abstract**: full text is overkill for a router and 5× the cost
- **Cheap model for routing**: Haiku here, Opus for the specialist
- **Log the input and the choice**: your router-quality dashboard depends on it

When to reach for which pattern

- Doc fits in context, single domain → **simple summarize**
- Long doc, single domain → **map-reduce**
- Mixed domains → **router + specialists**
- Output drives downstream work → **structured action-item template**
- Production reliability matters → **misclassification audit on top of all of the above**
- **These compose. Real systems use all five.**

Summarization

https://github.com/DataForScience/AIAutomation

Copyright 2026 Pearson. All rights reserved.

Table of Contents:

1 Structured Extraction

2 Summarization

3 Agents

4 Chatbot



Agents Project Overview

- Goal: Build a multi-agent crew that audits privacy policies against a GDPR / CCPA compliance checklist, runs sync first, then async to compare throughput.

- The crew
  - Researcher: reads the policy, extracts factual statements about data handling.
  - Compliance Officer: maps statements to a checklist of requirements; flags gaps.
  - Writer: assembles a clean, executive-readable audit report.

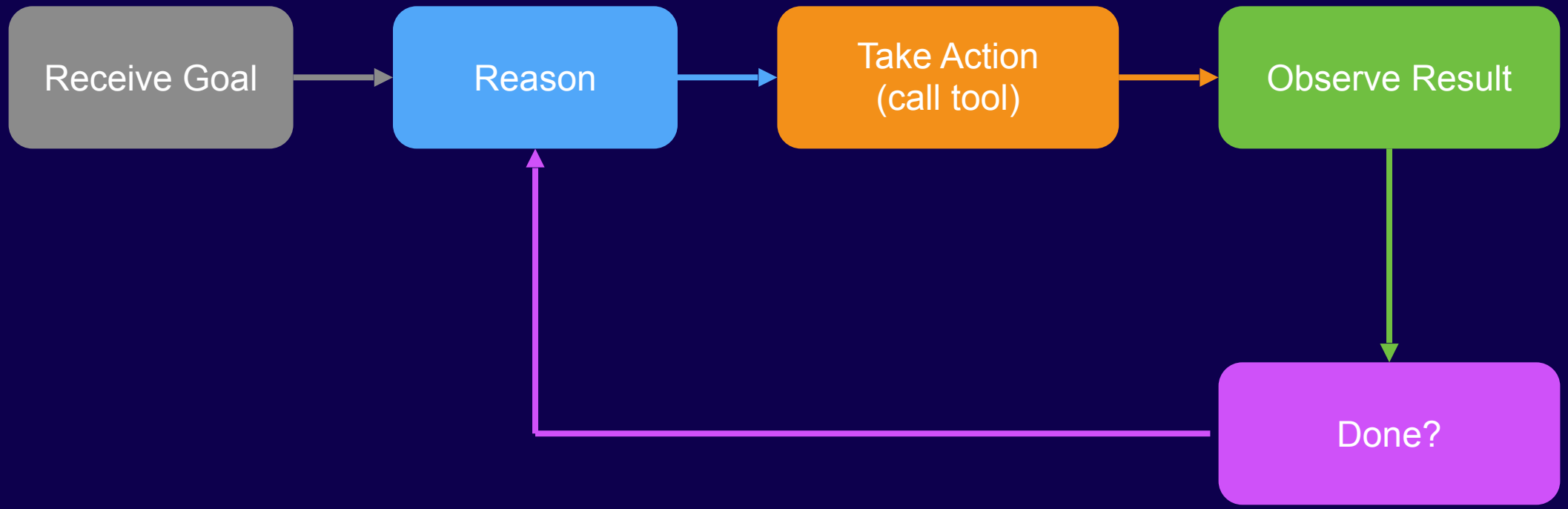
- Dataset: Live privacy policies fetched from a few well-known public sites.

⚠️ This is a *teaching* tool. The output is not legal advice. Real GDPR audits require a qualified DPO. ⚠️

Pearson

Agentic loop

- Short loop.
- The interesting work is the prompt that runs at every step.



Agent failure modes

- **Looping**: same tool, same args, no progress. Set **max_iterations**.
- **Wrong tool selection**: fix with sharper tool descriptions, not more system-prompt rules
- **Cost explosion**: every step is a full forward pass. Budget by step count and dollar.
- **Brittle hand-offs**: multi-agent systems amplify any single agent's format drift
- **Mitigations:** structured outputs at every hand-off, observability (LangSmith / traces), and a hard step budget

CrewAI Deep Dive

- **CrewAI** coordinates multiple LLM agents working on a shared goal
- Five primitives:
  - **Agent**: role, goal, backstory, LLM, tools
  - **Task**: description, expected_output, agent, context
  - **Crew**: agents + tasks + a process (sequential / hierarchical)
  - **Tool**: same tool model as LangChain
  - **Process**: execution policy: who runs when
- **docs.crewai.com** for the rest

CrewAI: Agent

- **Three rules of thumb:**
  - One responsibility per agent. "Researcher" extracts; "Officer" judges.
  - Explicit goal beats long backstory: goal steers behavior
  - Constrain output format in the Task, not the Agent: keeps agents reusable

```
researcher = Agent(
    role='Privacy Researcher',
    goal=('Extract every concrete factual statement '
          'about how data is collected, used, shared, '
          'retained, or protected in the given policy.'),
    backstory='You are a meticulous analyst who quotes the source. '
              'You never infer.',
    llm=claude_llm,
    verbose=False,
)
```

CrewAI: Task

```python
t1 = Task(
    description=(
        'Read the policy below and produce a numbered list '
        'of factual statements. Each item: 1-sentence '
        'paraphrase + a short verbatim quote in <quote>...</quote>.\n\n'
        f'POLICY:\n{policy_text}'
    ),
    expected_output='Numbered list, ~10–20 items.',
    agent=researcher,
)

t2 = Task(
    description='Map facts to the GDPR/CCPA checklist as JSON.',
    expected_output='Pure JSON — no prose around it.',
    agent=officer,
    context=[t1],     # <-- output of t1 feeds t2
)
```

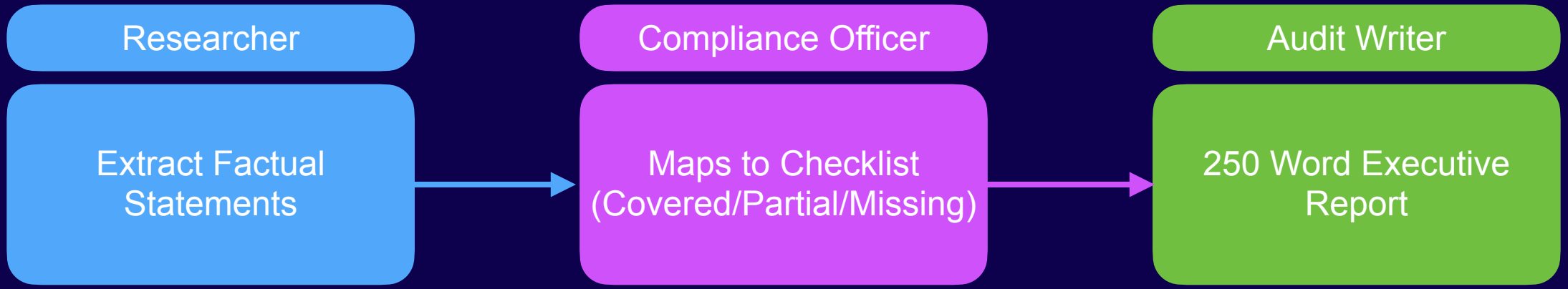
Pearson

CrewAI: Process

- **Process.sequential**: tasks run in order; output of N feeds N+1
  - best when the workflow is linear and well-understood
- **Process.hierarchical**: a manager agent decides who runs and revises
  - best when tasks need branching or revision loops
- **Start sequential. Promote to hierarchical only when you actually need the manager to make decisions.**

Researcher → Officer → Writer

- One responsibility per agent. Each hand-off is auditable.
- Sequential Pipeline. Each agent hands off to the next one on the list



Pearson

Why split into three agents?

- **Specialisation**: extraction prompts and judgment prompts have different failure modes
- **Auditability**: you can read the researcher's output before the officer judges. In a regulated workflow that's the whole point.
- **Cost control**: researcher can run on Haiku, officer on Sonnet, writer on Opus
- **Replaceability**: swap the writer for one that produces JSON for Jira, or a Slack message, without touching extraction

Compliance crew: the data

- Live-fetched public privacy policies (3–4 docs, ~12 KB each)
  - GitHub, DuckDuckGo, Wikipedia, …
- 8-item GDPR/CCPA checklist (real audits use 100+; this is the demo subset)
  - purpose_limitation, data_minimization, retention_period, third_party_sharing, user_rights, contact_dpo, legal_basis, ccpa_opt_out

⚠️ This is a *teaching* tool. The output is not legal advice. Real GDPR audits require a qualified DPO. ⚠️

Synchronous vs Asynchronous crews

- **Sync**: one crew at a time. Easy to reason about, easy to debug.
- **Async**: many crews in parallel via **asyncio.gather** + **asyncio.to_thread**
- **Most crews are I/O bound: they spend most time waiting on the LLM API**
  - Async over independent inputs is essentially free throughput
  - **Be careful with rate limits: Anthropic's tier matters here**

Async crews: the pattern

- CrewAI's kickoff is sync; this lets us run N of them concurrently without rewriting the framework

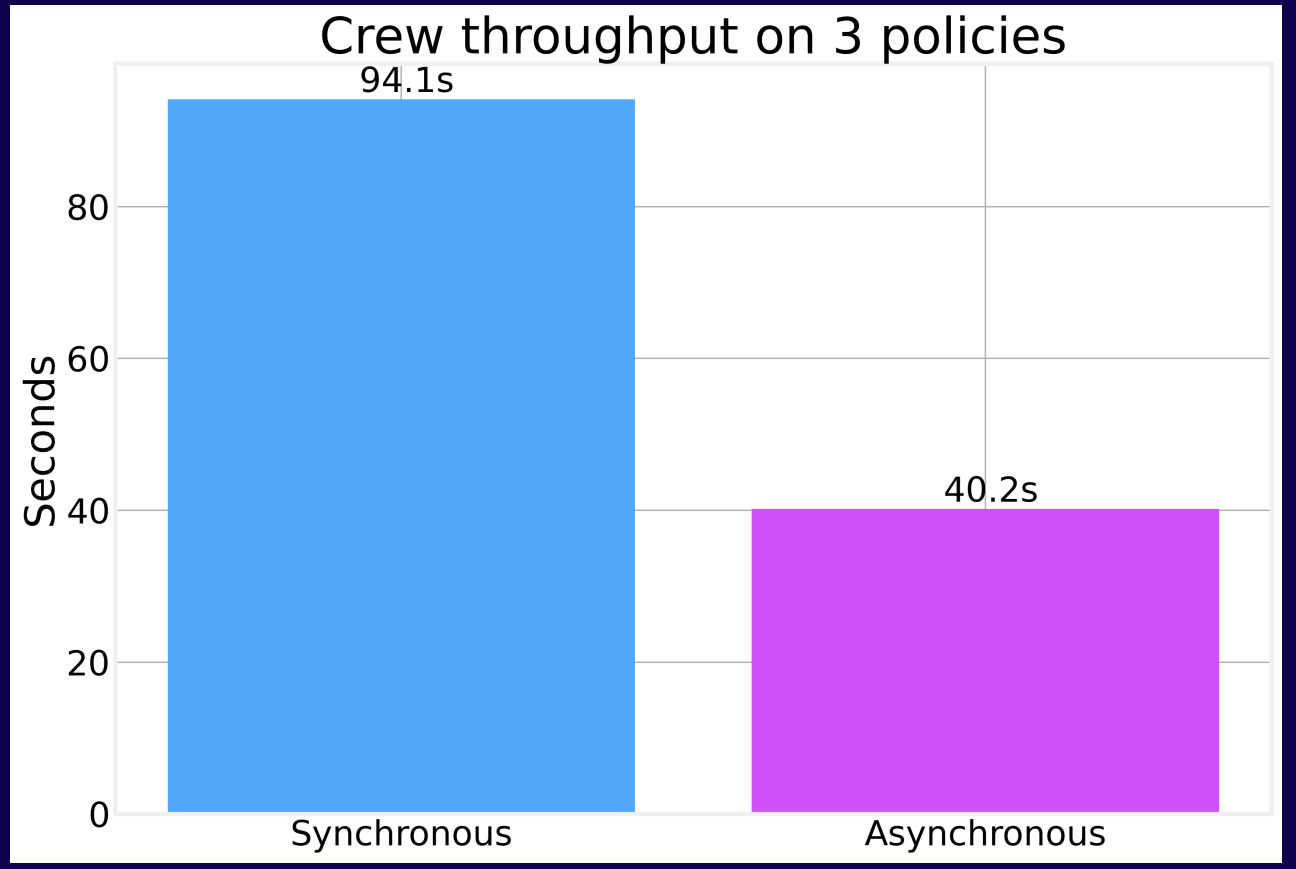
```python
def audit_sync(name, text):
    crew = Crew(
        agents=[researcher, officer, writer],
        tasks=build_tasks(text),
        process=Process.sequential,
    )
    return crew.kickoff()

# wrap the sync call in a thread; gather across inputs
async def audit_all_async(items):
    async def one(name, text):
        return name, await asyncio.to_thread(audit_sync, name, text)
    return dict(await asyncio.gather(
        *[one(n, t) for n, t in items.items()]
    ))
```

Pearson

Sync vs async: throughput

- ~2× speed-up with no logic changes.



Pearson

Compliance checklist creation

- The Officer's output is structured JSON, not prose:

```
{
  "purpose_limitation":   {"status": "covered",  "evidence": "Section 2..."},
  "data_minimization":    {"status": "partial",  "evidence": "..."},
  "retention_period":     {"status": "missing",  "evidence": ""},
  "third_party_sharing":  {"status": "covered",  "evidence": "Section 5..."},
  "user_rights":          {"status": "covered",  "evidence": "Section 6..."},
  "contact_dpo":          {"status": "covered",  "evidence": "footer"},
  "legal_basis":          {"status": "missing",  "evidence": ""},
  "ccpa_opt_out":         {"status": "partial",  "evidence": "..."}
}
```

- That JSON is the artifact.
- The Writer turns it into an executive report.
- Both are useful downstream. JSON for dashboards, report for humans.

Pearson

Where this goes next

- Add a **Reviewer** agent that re-reads the writer's report and contests it. Feed disagreements back via **Process.hierarchical**.
- Connect the JSON output to a real ticket system
- Run the crew on a schedule. Diff this week's checklist against last week's.
- Replace the demo checklist with the real GDPR one

Agents

https://github.com/DataForScience/AIAutomation

Copyright 2026 Pearson. All rights reserved.

Table of Contents:

1 Structured Extraction

2 Summarization

3 Agents

4 Chatbot



Chatbot Project Overview

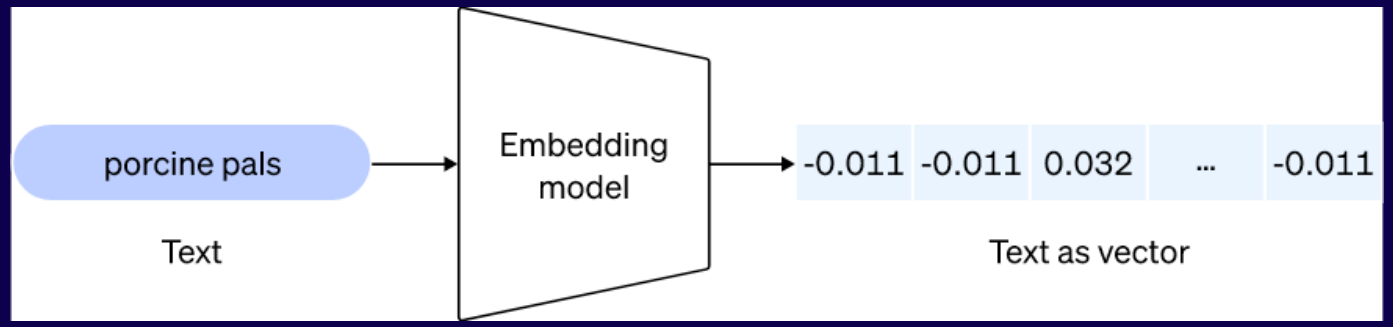
- Goal: Build a chatbot that answers questions over a Wikipedia subset using RAG, plus tool use for things RAG can't do (live calculation, current time).

- Stack:
  - Embeddings: sentence-transformers/all-MiniLM-L6-v2 (~80 MB, CPU-friendly)
  - Vector DB: ChromaDB (in-memory; no server)
  - LLM: Claude via LangChain
  - Tools: a calculator, a clock, and the retriever itself

- Dataset: https://huggingface.co/datasets/rag-datasets/rag-mini-wikipedia)

Embeddings

- An **embedding** maps text to a dense numerical vector
- Trained so similar meanings get similar vectors
- Lets us measure semantic similarity by vector distance
- Embeddings power semantic understanding



Embedding providers

- **Voyage AI**: Anthropic's recommended partner. **voyage-3.5-lite** at $0.02 / 1M tokens.
- **OpenAI**: **text-embedding-3-large**, widely supported
- **Cohere embed-v3**: strong multilingual
- **Open weights: sentence-transformers/all-MiniLM-L6-v2** runs locally on CPU.
- All produce dense float vectors. Pick one, stick with it: re-embedding is expensive.

Vector databases

- Specialized stores that retrieve by semantic similarity, not exact match
- Common choices:
  - **Pinecone**: managed cloud, fast at scale
  - **Weaviate**: open source, hybrid search out of the box
  - **ChromaDB**: local/embedded, perfect for the local demos
  - **pgvector**: Postgres extension, often the right answer for small-to-medium scale
- **Don't reach for a vector DB until you have ≥10K vectors. NumPy + cosine works fine below that.**

ChromaDB: minimal example

```python
import chromadb
from chromadb.config import Settings

client = chromadb.Client(Settings(anonymized_telemetry=False))
collection = client.create_collection("wiki")

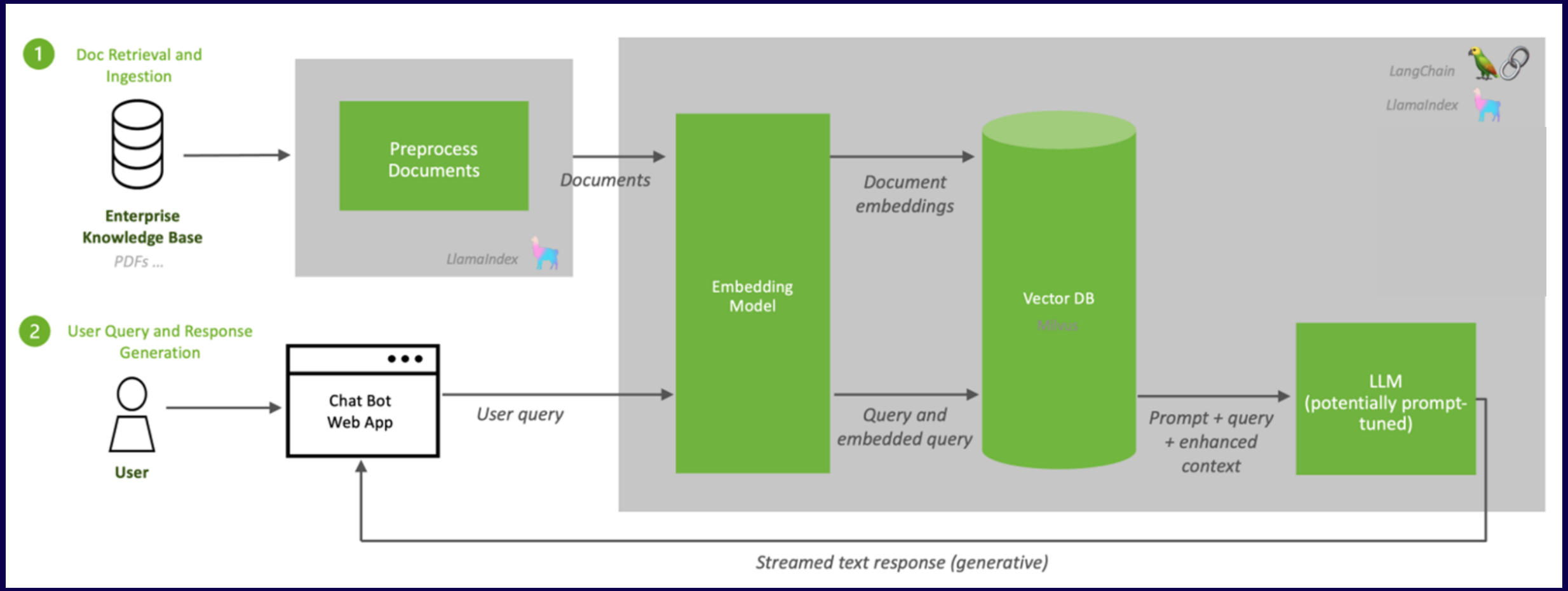
collection.add(
    ids=[str(i) for i in range(len(passages))],
    embeddings=embeddings.tolist(),     # (N, d) float matrix
    documents=passages,
)

# query
qv = embedder.encode(["Who wrote Hamlet?"])
res = collection.query(
    query_embeddings=qv.tolist(),
    n_results=4,
)
print(res["documents"][0])              # top-4 passages
```

Pearson

Retrieval Augmented Generation

- Combine Search and LLMs for Question Answering



https://developer.nvidia.com/blog/tips-for-building-a-rag-pipeline-with-nvidia-ai-langchain-ai-endpoints/

RAG: prompt is all you need

```python
RAG_PROMPT = ChatPromptTemplate.from_messages([
    ("system",
     "Answer using ONLY the numbered passages below. "
     "Cite each fact as [#] referring to the passage number. "
     "If the passages do not contain the answer, "
     "say so explicitly."),
    ("user",
     "Question: {question}\n\nPassages:\n{ctx}"),
])

def answer(question):
    passages = retrieve(question, k=4)
    ctx = "\n\n".join(f"[{i+1}] {p}"
                      for i, p in enumerate(passages))
    return (RAG_PROMPT | llm | StrOutputParser()).invoke({
        "question": question, "ctx": ctx,
    })
```

Pearson

Guardrails

- **"Answer using ONLY the passages"**: refuses to fall back on the model's prior knowledge
- **"If the passages don't contain the answer, say so"**: kills the most common RAG hallucination
- **Citation by passage number, not URL**: harder for the model to fabricate citations
- **Number the passages explicitly**: "[1]" "[2]" "[3]": gives the model a stable referent
- **Eval against your own gold Q&A set every time you change the prompt**

RAG quality is bounded by retrieval quality

- If the right passage isn't in the top-K, no prompt can save you
- **Reranking:** pull top-50 with embedding search, rerank with a cross-encoder, keep top-5
- **Hybrid search:** combine BM25 (lexical) with vector (semantic).
- **HyDE:** have the LLM generate a hypothetical answer, embed that, search with it. Often the cheapest +5 points.
- **Query expansion:** generate 3 paraphrases, search them all, dedupe

Tool integration

- RAG is great for facts in a corpus. **Awful for arithmetic and time.**
- Tools fill those gaps:
  - **Calculator**: for any math the model would otherwise guess at
  - **Clock / date**: for "today", "now", "yesterday"
  - **Wiki search (the retriever as a tool)**: model decides when to retrieve
  - **Anything with an API**: weather, stocks, internal services
- The model sees only typed names + descriptions + JSON schemas: keep them sharp.

Tools: the @tool decorator

```python
@tool
def calculator(expression: str) -> str:
    """Evaluate a Python arithmetic expression.
    Use for any math the user asks for."""
    try:
        allowed = {"__builtins__": {}}
        return str(eval(expression, allowed,
                        {"abs": abs, "min": min, "max": max,
                         "round": round}))
    except Exception as e:
        return f"error: {e}"
```

Tool design

- **Specificity**: **get_weather(city, country)** beats **get_external_data(query)**
- **Descriptions**: the docstring is the prompt. "Use for any math" tells the model when to call it.
- **Schema design**: required fields, **Literal** enums, descriptions on every parameter
- **Idempotence**: tool calls get retried. Make them safe to call twice.
- **Observability**: log every tool call. You'll thank yourself later.

Interactive agent: the loop

```python
llm_with_tools = llm.bind_tools(tools)
tool_map = {t.name: t for t in tools}

def chat(question, max_loops=5, history=None):
    history = history or [SystemMessage(content=SYSTEM)]
    history.append(HumanMessage(content=question))
    for _ in range(max_loops):
        ai = llm_with_tools.invoke(history)
        history.append(ai)
        if not getattr(ai, "tool_calls", None):
            return ai.content, history
        for call in ai.tool_calls:
            result = tool_map[call["name"]].invoke(call["args"])
            history.append({
                "role": "tool",
                "tool_call_id": call["id"],
                "content": str(result),
            })
    return "[max loops reached]", history
```

Pearson

What it looks like in practice

- **Q:** Who wrote Hamlet?
  - → wiki_search("Hamlet author") → "Shakespeare wrote Hamlet around 1599-1601 [3]"
- **Q:** What is 217 * 84 + 5?
  - → calculator("217*84+5") → “18233"
- **Q:** Tell me about Apollo 11, then convert 384,400 km to miles.
  - → wiki_search(...) → calculator("384400 / 1.609")
  - → answers both, in order
- The model **decides which tool**, when, in what order.
- Your job is to design **good tools**

Key takeaways

- **Schema is contract.** Pydantic gives you validation, parsing, and a JSON Schema for free.
- **Validation loops cost cents and double reliability.** One self-repair attempt covers most format failures.
- **Routers are cheap insurance.** Sub-cent classifications gate expensive specialists.
- **Multi-agent ≠ better.** Reach for a crew when the workflow is auditable, not for prestige.
- **RAG quality ≤ retrieval quality.** Optimise retrieval before tweaking prompts.
- **Tools are the sharpest abstraction in the stack.** Treat the docstring as the prompt.
- **Default to the simplest pattern that works. Add complexity only when something breaks.**

ChatBot

https://github.com/DataForScience/AIAutomation

Copyright 2026 Pearson. All rights reserved.

Contacts



Bruno Gonçalves is an author, public speaker, corporate trainer, and consultant specializing in Generative AI, Blockchain Analytics, and Machine Learning. He founded Data For Science, Inc in 2009 to help individuals and companies solve their data driven problems.

Bruno Gonçalves

https://data4sci.com

info@data4sci.com

https://data4sci.com/call

https://www.linkedin.com/in/bmtgoncalves/